

TeamCore

Company Operations, Workforce, Project & Task Management Platform

Primary stack: Django • Django REST Framework • PostgreSQL • Django Templates • HTML/CSS/JavaScript • Tailwind CSS • Flutter

Scale target: approximately 160 employees initially, designed for 1,000+ users.

Architecture: Django-based modular monolith with a shared PostgreSQL database and Flutter/Web clients.

1. Executive Summary

TeamCore is a centralized internal company platform for managing employees, departments, teams, projects, tasks, QA, bugs, productivity, attendance, leave, notifications, reports and audit history.

The platform is designed around one operational chain: Employee → Team → Project → Task → Progress → Review → QA → Completion → Analytics. Management should be able to see what is happening across the organization without manually collecting status from multiple tools.

2. Product Goals

- Create a single source of truth for company execution.
- Assign work clearly and monitor progress and deadlines.
- Detect overdue and blocked work early.
- Give PMs and HODs workload and project-risk visibility.
- Give QA a structured review and bug workflow.
- Give HR centralized employee, attendance and leave management.
- Maintain complete audit history for important actions.
- Provide a scalable foundation for 1,000+ users.

3. Roles & Permission Model

Role	Main responsibilities
ADMIN	System-wide administration, users, permissions, configuration, audit and reports.
HR	Employee records, attendance, leave, organizational data and HR reports.
PROJECT_MANAGER	Projects, milestones, teams, task assignment, deadlines and project health.
COT / HOD	Department oversight, workload, team performance and project risks.
EMPLOYEE	Assigned tasks, progress, comments, files, time records, attendance and leave.
QA	QA queue, test results, bugs, verification and rework.
CLIENT	Restricted access to permitted projects, milestones and approved deliverables.

Use RBAC plus granular permissions such as task.assign, task.review, task.qa, project.manage, employee.manage and report.view. Permissions must always be enforced on the Django server.

4. Organization & Employee Management

- Employee ID, name, avatar, department, team, designation, role and manager/HOD.
- Joining date, contact details, skills and employment status.
- Project membership and task history.
- Employee lifecycle: invited → active → leave/suspended → inactive.
- Department → Team → Employee hierarchy.
- Historical relationships should remain auditable after transfers.

5. Project Management

- Project manager, client, team members, dates, priority and status.
- Milestones and target dates.
- Tasks, dependencies, files, comments and activity history.
- Project status: PLANNING, ACTIVE, ON_HOLD, AT_RISK, COMPLETED, CANCELLED.
- Project health based on deadline, overdue tasks, blockers, velocity and QA signals.

6. Advanced Task Management

Task is the core operational entity.

Field	Purpose
Project / Parent Task	Project relationship and optional hierarchy.
Assignee / Reviewer / QA	Ownership and review responsibility.
Priority / Status	Execution priority and workflow state.
Start / Due Date	Planning and deadline tracking.
Estimated / Actual Hours	Planning and effort analysis.
Progress %	Visible work completion indicator.
Comments / Attachments	Collaboration and evidence.
Dependencies	Controls work order and blockers.
Activity History	Accountability and audit.

Recommended workflow:

BACKLOG → TODO → IN_PROGRESS → BLOCKED → IN_REVIEW → QA → REWORK → COMPLETED

The backend should validate legal status transitions instead of trusting client-supplied status values.

7. Workload, Capacity & Assignment

- Before assignment, PM can see active tasks, estimated hours, deadlines and current workload.
- Show planned hours versus weekly capacity.
- Highlight overloaded employees as planning signals.
- Support team-level workload views.
- Do not use workload alone as an automatic performance judgment.

8. Blockers & Dependencies

- Employee can mark a task BLOCKED.
- Block reasons: client, design, API, requirement, dependency, technical issue or other.
- Record blocked start/end times.
- Support task dependency chains such as Design → Frontend → Backend → QA → Release.
- Blocked duration should appear in project-risk analytics.

9. QA & Bug Management

- Developer submits work → QA queue → PASS or FAIL.
- FAIL moves work to REWORK with QA feedback.
- Bug fields: project, task, reporter, developer, QA, severity, priority, environment, steps, expected/actual result, evidence and resolution.
- Severity: CRITICAL, HIGH, MEDIUM, LOW.
- Bug lifecycle: REPORTED → ASSIGNED → IN_PROGRESS → FIXED → QA → VERIFIED / REOPENED.

10. Productivity & Time Tracking

- Tasks completed and completed on time.
- Overdue and blocked work.
- Estimated versus actual effort.

- Review/rework rate.
- QA pass rate.
- Project contribution.
- Optional task timer: Start → Pause → Resume → Stop.
- Prefer outcome-based productivity indicators over invasive surveillance.

11. Attendance & Leave

- Check-in/check-out and attendance status.
- Late/early indicators.
- Leave types, requests, approvals and holiday calendar.
- Approval workflow: Employee → Manager/HOD → HR policy validation → Approved/Rejected.
- Restrict attendance and HR information by permission.

12. Notifications & Collaboration

- Task assignment, deadline reminder, overdue, QA result, project change, mention and approval notifications.
- In-app notifications plus Flutter push and optional email.
- Task/project comments with @mentions.
- Central notification history and read/unread state.

13. Dashboards

Dashboard	Key information
Company	Employees, projects, tasks, overdue, blockers, at-risk projects, department trends.
Employee	My tasks, deadlines, progress, blockers, notifications, attendance/leave.
Project Manager	Project health, milestones, team workload, overdue/blockers, QA and forecast.
HOD	Department workload, team performance signals, risks and trends.
QA	QA queue, bugs by severity, pass/fail, reopens and release readiness.

14. Reporting & Analytics

- Project health and delivery variance.
- Task completion and overdue analysis.
- Blocked-work duration.
- Workload/capacity.
- QA pass/rework and bug trends.
- Attendance and leave reports.
- Department and team trends.
- Authorized CSV/XLSX/PDF export.

15. AI Features — Future Phase

- AI task-effort estimation using historical completed tasks.
- Project risk prediction from overdue, blocker, velocity and QA signals.
- Suggested assignee based on skills, workload and project context.
- AI-generated weekly project summaries.
- Anomaly detection in delivery/workload patterns.
- Natural-language management queries.
- AI should recommend; authorized managers retain final decision authority.

16. Flutter Mobile App

- Authentication and secure token storage.
- Employee: Dashboard, My Tasks, Task Detail, Progress, Timer, Comments, Files, Notifications, Attendance, Leave.
- Manager: Projects, Team, Task Board, Assignment, Approvals, Reports.
- QA: QA Queue, Test Detail, Bug creation/editing, Re-test and Verification.
- Use API pagination and push notifications.

17. Web Frontend

Use Django Templates with HTML, Tailwind CSS and JavaScript. React is not required for this internal Django-centric system.

Reusable UI components should include sidebar, navbar, cards, tables, filters, forms, modals, status badges, charts, pagination and permission-aware actions.

Use JavaScript only where dynamic interaction improves the server-rendered experience.

18. Django Architecture

```
backend/ ├── manage.py ├── config/ ├── core/ ├── accounts/ ├── departments/ ├── projects/ ├── tasks/ ├── qa/ ├── productivity/ ├── attendance/ ├── notifications/ ├── reports/ ├── files/ ├── audit_logs/
```

Start as a modular monolith. Each domain has its own models, services, URLs and tests where appropriate. Extracting services later is possible if actual scale requires it.

19. PostgreSQL Core Models

Model	Purpose
User	Authentication, employee identity, role and status.
Department / Team	Organization hierarchy.
Project / ProjectMember	Projects and team membership.
Milestone	Project milestones.
Task / TaskDependency	Work and dependency graph.
TaskComment / Attachment	Collaboration and evidence.
TimeEntry	Actual effort records.
QARecord / Bug	Quality workflow.
Attendance / LeaveRequest	HR operations.
Notification	In-app/push/email state.
AuditLog	Immutable critical activity history.

20. API Design

Use Django REST Framework with a versioned prefix such as `/api/v1/`.

Domain	Example endpoints
Auth	POST <code>/api/v1/auth/login</code> • <code>/refresh</code> • <code>/logout</code>
Employees	GET <code>/api/v1/employees</code> • <code>/id</code>

Domain	Example endpoints
Projects	GET/POST /api/v1/projects • PATCH /{id}
Tasks	GET/POST /api/v1/tasks • PATCH /{id}
Task Actions	POST /api/v1/tasks/{id}/assign • /progress • /status
QA	GET /api/v1/qa/queue • POST /api/v1/qa/{id}/verify
Reports	GET /api/v1/reports/project-health • /workload • /productivity

21. Security

- Django password hashing and production security settings.
- Server-side RBAC and object-level authorization.
- HTTPS in production.
- Secrets in environment/configuration, never Git.
- CSRF protection for web forms and secure API authentication for Flutter.
- Rate limiting for sensitive endpoints.
- Upload type/size validation and controlled media access.
- Audit security-relevant actions without logging secrets.
- PostgreSQL backup and tested restore process.

22. Scalability for 1,000+ Users

- Index common filters: department_id, team_id, project_id, assignee_id, status, due_date, created_at.
- Use pagination for all large lists.
- Use Redis caching where profiling proves it useful.
- Use Celery + Redis or equivalent background jobs for reminders, notifications and heavy reports.
- Use transactions/locking for race-sensitive task assignment and workflow changes.
- Use object/media storage appropriate for production.
- Monitor API latency, DB load, worker queues, errors and storage.

23. Background Jobs

- Overdue detection.
- Deadline reminders.
- Push/email notifications.
- Daily/weekly summaries.
- Report generation.
- Scheduled analytics aggregation.
- File processing.
- Future AI analysis.

24. Audit & History

Every important action should record actor, action, object, timestamp and relevant before/after values.

Example: User A assigned Task #102 → User B changed TODO to IN_PROGRESS → QA rejected → task moved to REWORK → QA approved.

Audit history should be searchable by employee, project, task, action and date.

25. Testing Strategy

- Unit tests for models, services, permissions and business rules.
- API tests for authentication, validation and authorization.
- Workflow tests for task and QA transitions.
- Concurrency tests for assignment/state changes.
- Permission matrix tests for every role.
- Flutter integration tests for core flows.
- Performance tests for dashboards, lists and reports.
- Backup/restore and deployment tests.

26. Deployment

- Development, staging and production environments.
- Django behind a production reverse proxy/web server.
- PostgreSQL as primary database.
- Redis for cache/background jobs when needed.
- Worker process for asynchronous tasks.
- Secure static/media storage.
- CI/CD after the project stabilizes.
- Production migration and rollback strategy.

27. Development Roadmap

Phase	Scope
1. Foundation	Django config, custom User, roles, PostgreSQL, environment setup, base UI.
2. Organization	Departments, teams, employee profiles, manager/HOD relationships.
3. Projects	Projects, members, milestones and project dashboard.
4. Tasks	CRUD, assignment, board/list, status, deadlines, comments, dependencies.
5. QA	QA queue, bugs, review/rework and verification.
6. Productivity	Time entries, workload and productivity metrics.
7. HR	Attendance, leave, holidays and approvals.
8. Notifications	In-app, push, email and scheduled reminders.
9. Reports	Company, department, project, task, QA and productivity reports.
10. Flutter	Employee first, then manager and QA mobile workflows.
11. Hardening	Security, performance, backup, monitoring and audit verification.
12. Pilot	Deploy to one/two teams, collect feedback, refine, then roll out.

28. MVP Scope

- Custom User + RBAC.
- Departments, teams and employees.
- Projects and project members.
- Tasks, assignment, status, deadlines, comments and dependencies.
- Basic QA and bug workflow.
- Notifications.
- Company/project/employee dashboards.
- Audit logs.

- PostgreSQL + REST API.
- First Flutter employee workflow.

29. Recommended Future Modules

- Client portal.
- Resource/capacity planning.
- Project budget and cost tracking.
- Knowledge base.
- Internal announcements.
- Asset/device management.
- Recruitment/onboarding.
- Performance review cycles.
- Advanced AI analytics.
- Third-party integrations such as email/calendar/chat systems.

30. Final Architecture

Flutter Mobile ↓ Django REST API ↓ Django Business Logic / Services ↓ PostgreSQL ↑ Django Templates + Tailwind + JavaScript | Redis / Background Workers | Notifications / Reports / Future AI

Django remains the central source of business rules and data. Flutter and the web interface are clients of the same platform. This keeps the initial system maintainable while leaving a clear path for future scaling.

31. Success Criteria

TeamCore should allow management to answer five questions from one place: Who is working on what? What is late? What is blocked? Which projects are at risk? What needs management attention today?

The platform should reduce manual status collection, improve accountability, expose blockers earlier, provide reliable project history and create a scalable operational foundation for 1,000+ users.

End of TeamCore documentation.